

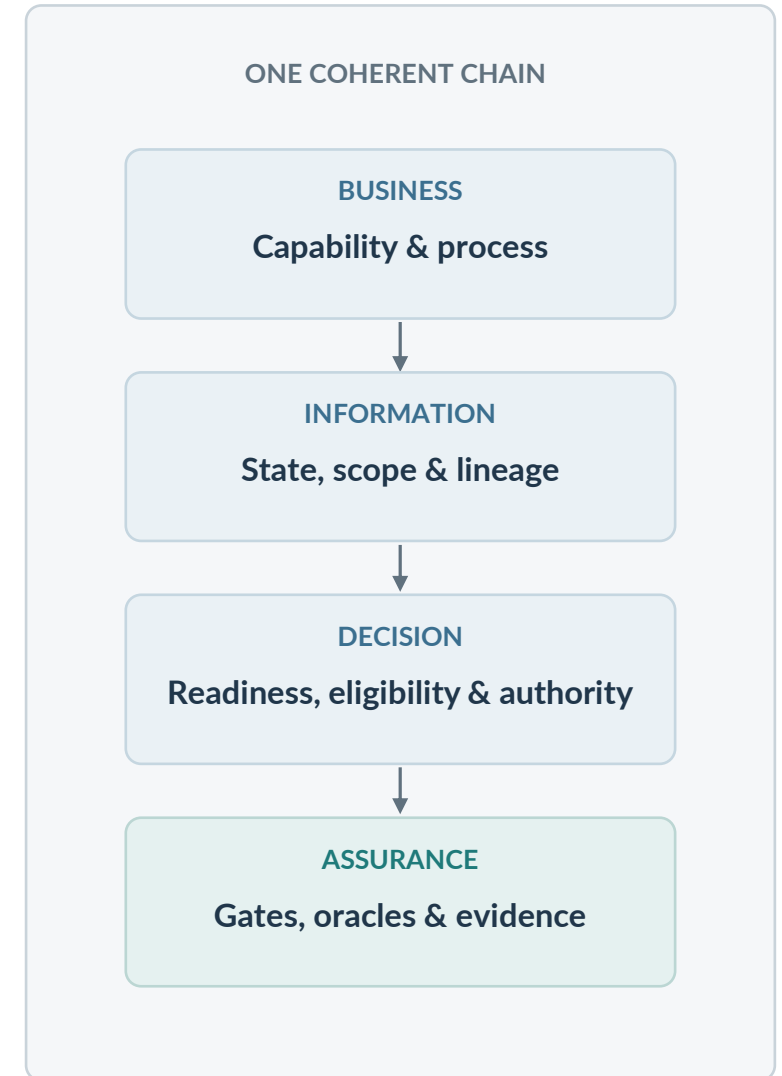
Product Change Impact Assessment & Decision Readiness

A product change becomes decision-ready only when current state, proposal, discovered impacts, domain conclusions, evidence, decision scope and authority are kept separate, and linked by deterministic, case-local and historically reconstructable rules.

Architecture first. Prototype as proof.

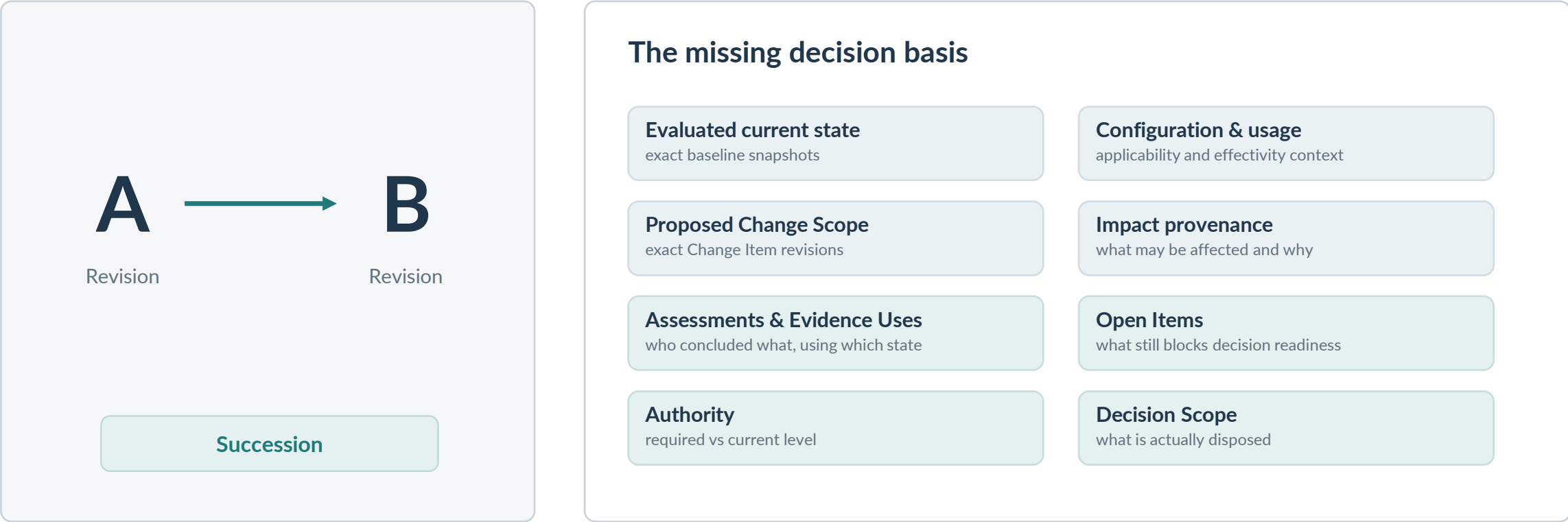
The implementation exists to test whether the frozen business, information and decision semantics execute without semantic shortcuts.

Cristian Leu · github.com/CmdrFALCO/plm-change-impact-reference-case



Why Revision A -> Revision B is insufficient

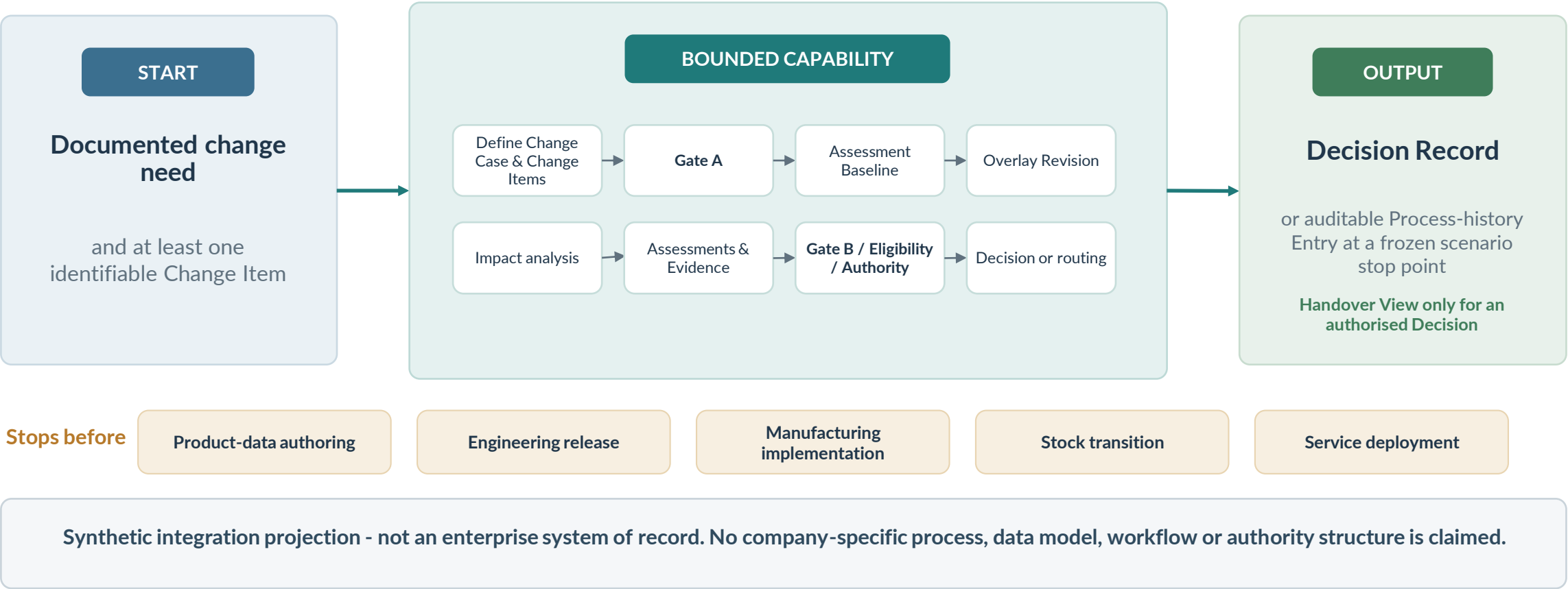
Revision succession records a relationship between two states. It does not identify the complete basis that was evaluated or authorised.



Revision succession is useful, but it is not a decision basis.

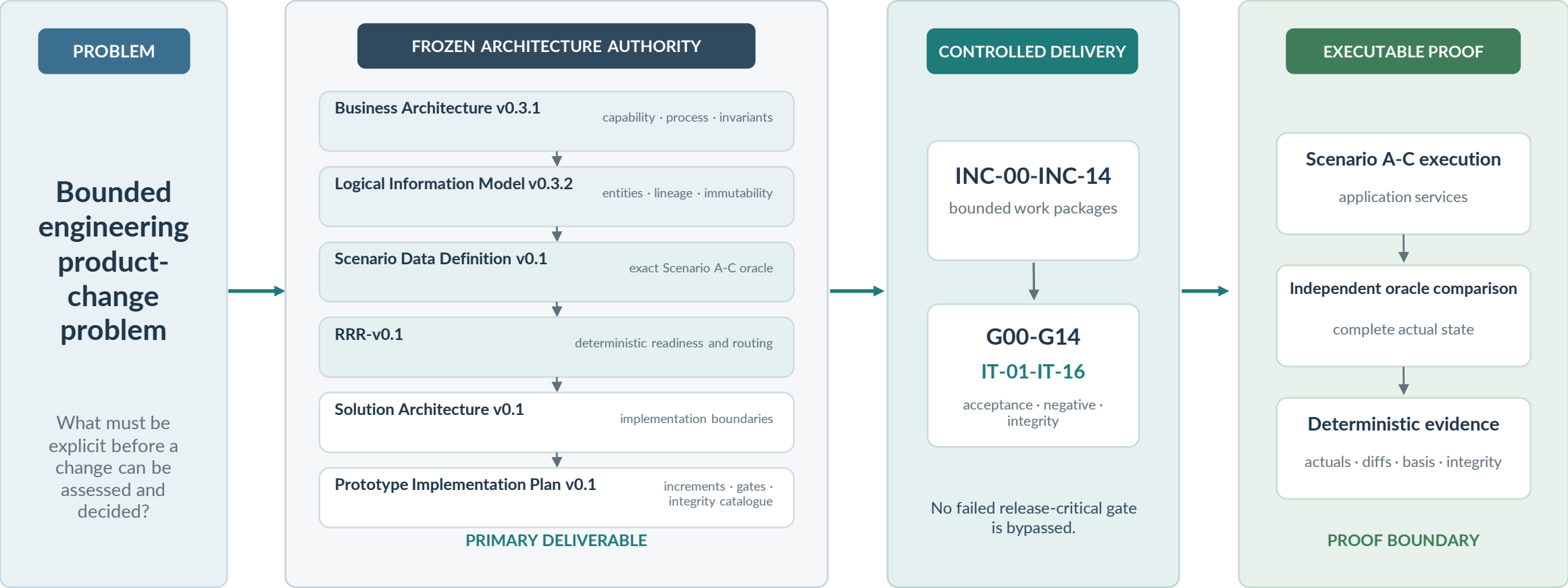
Bounded capability and public-safe scope

The reference case is deliberately narrow: enough scope to make decision readiness explicit, but no downstream implementation or company-specific fidelity claim.



From engineering problem to executable evidence

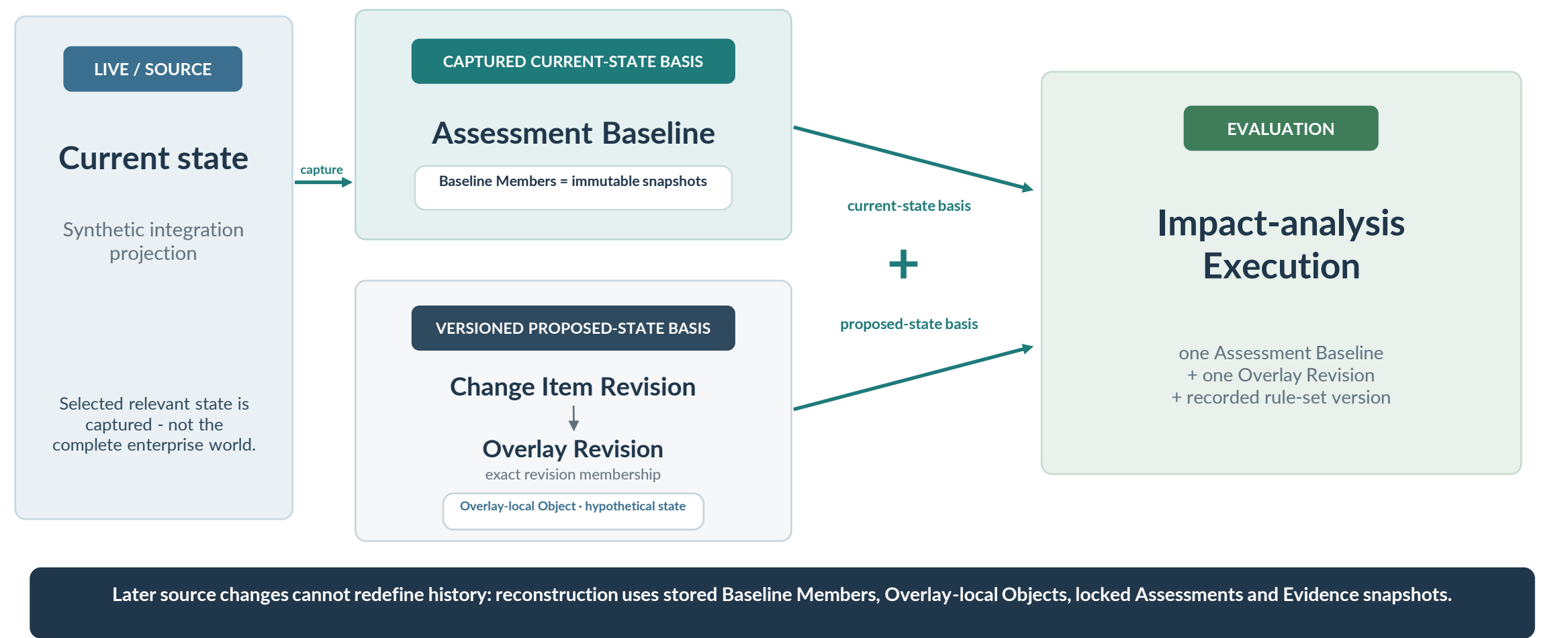
Business meaning governs delivery. Implementation, tests and evidence form the proof boundary.



Architecture is the primary deliverable. The prototype tests whether that architecture executes deterministically.

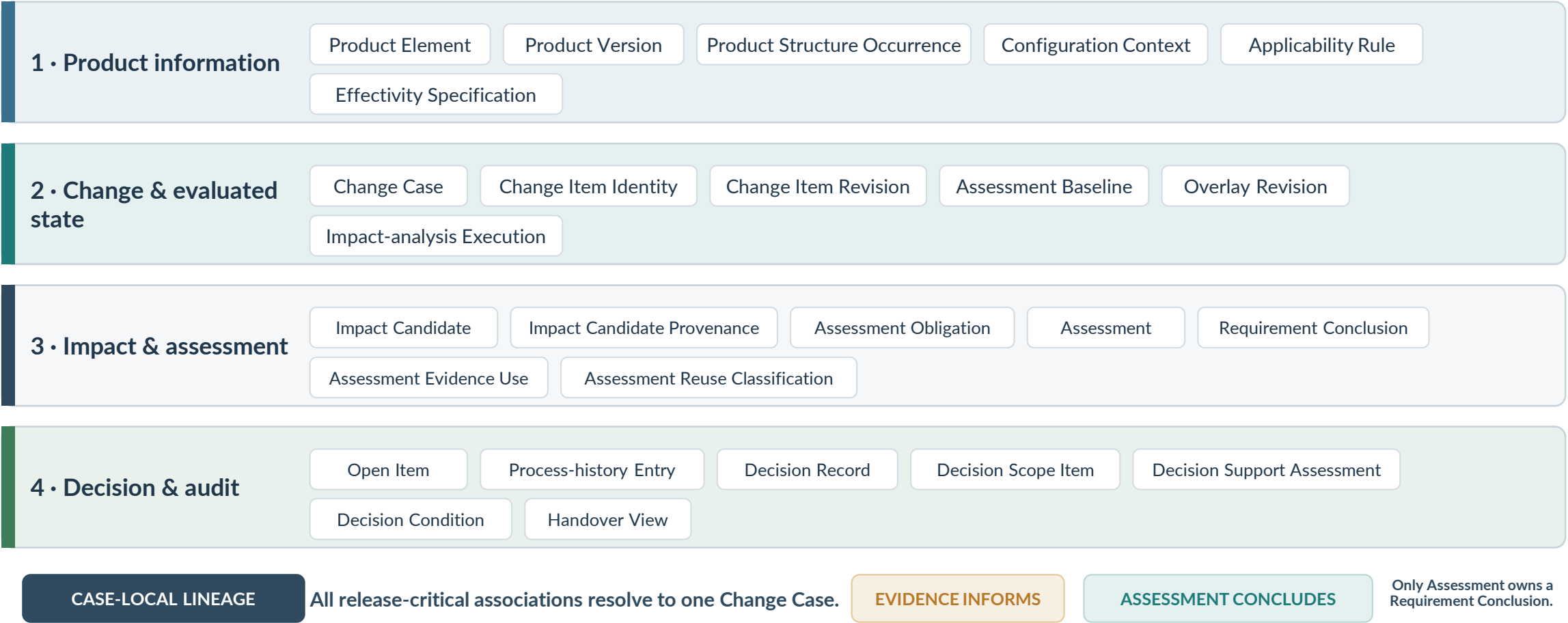
Current state, baseline and proposal are not equivalent

Historical evaluation requires an immutable captured basis and a separate, non-authoritative proposed-state overlay.



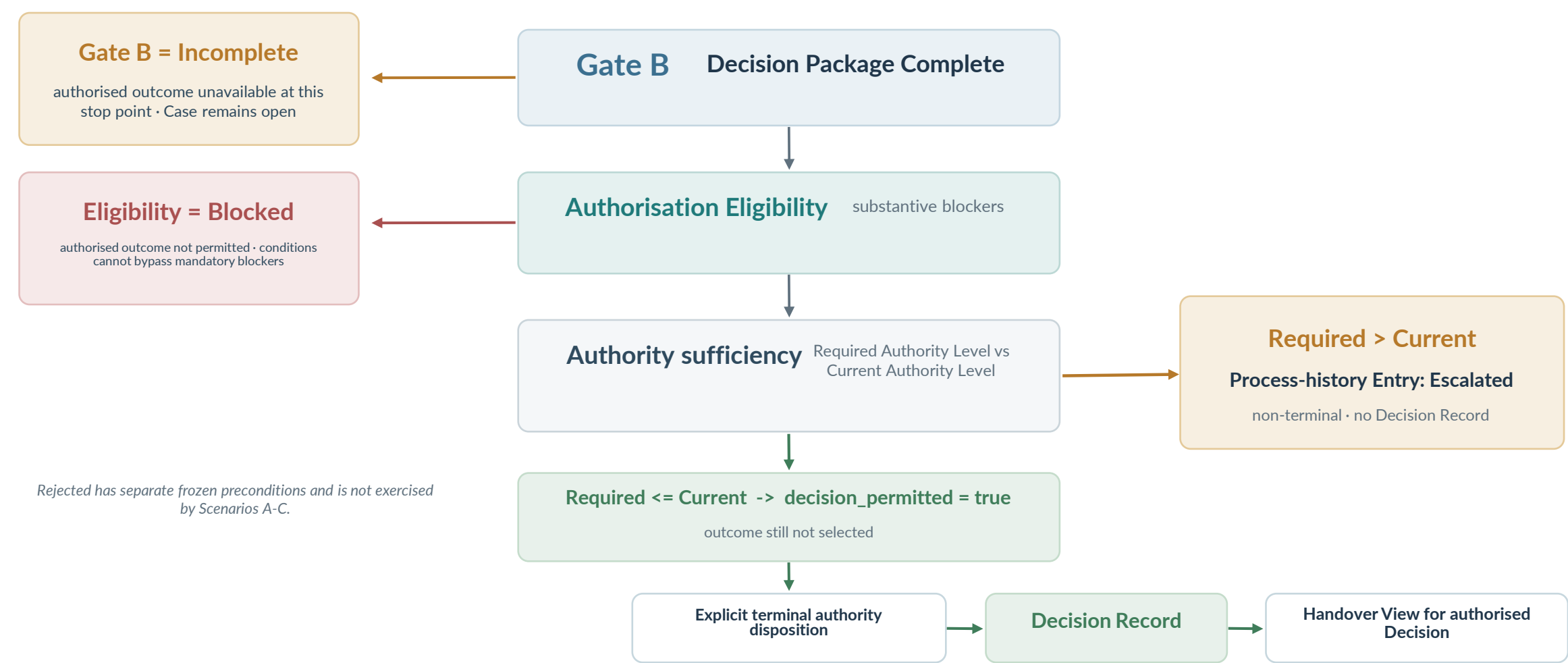
Information and lineage architecture

Decision readiness depends on explicit objects, associations and case-local lineage, not on one generic change record.



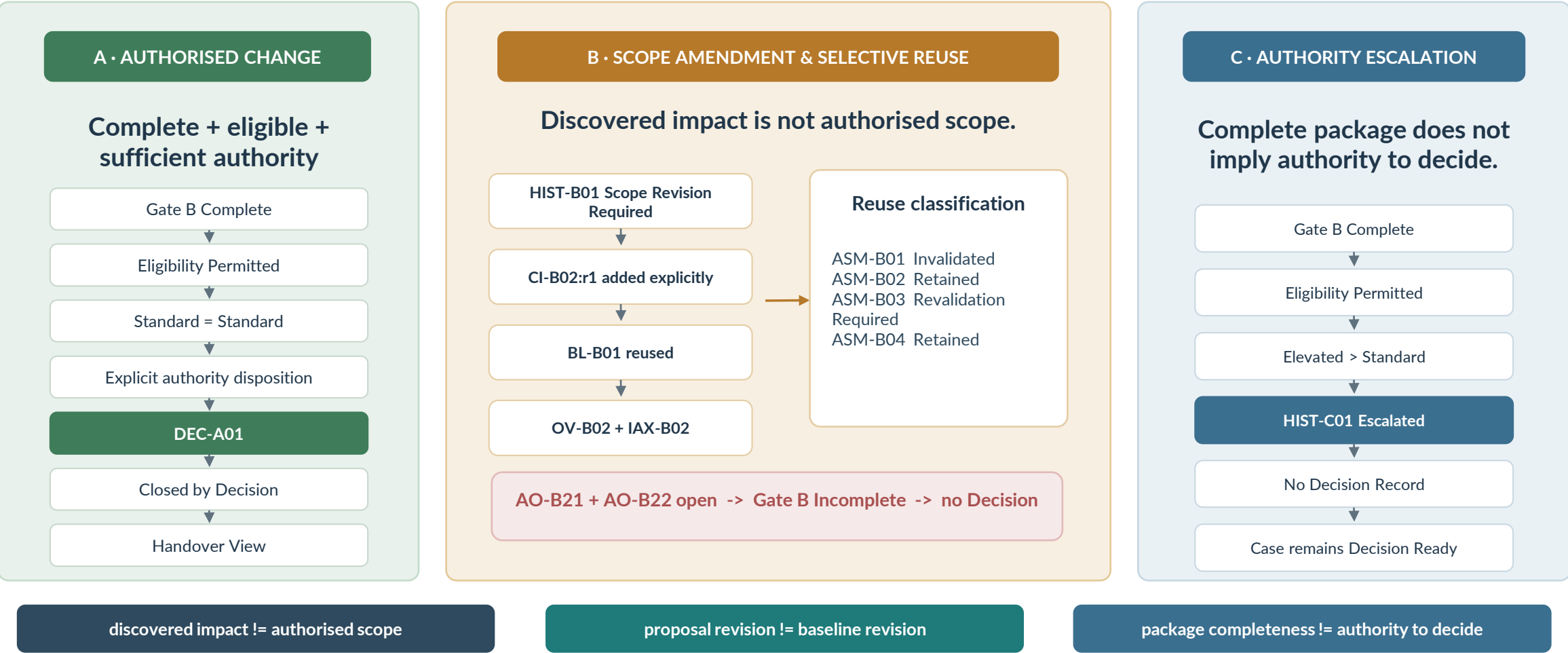
Decision readiness is not one gate

Package completeness, substantive eligibility, authority sufficiency and terminal disposition answer different questions.



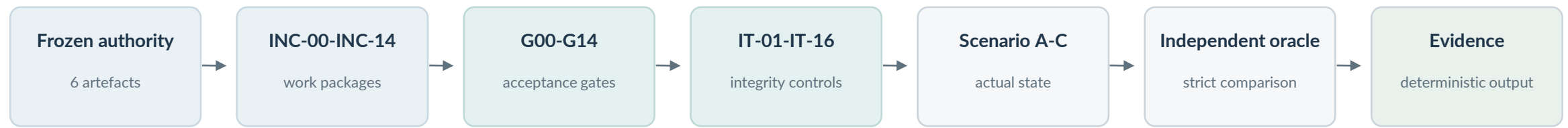
Scenarios A-C: three architectural proofs

The scenarios are independent fixtures that exercise approval, controlled scope revision and non-terminal authority escalation.



Controlled delivery and verification

The delivery chain is bounded, incremental and auditable: frozen authority -> work packages -> gates -> independent comparison -> evidence.



Three fixture roles remain separate

Scenario input

!=

Impact-result fixture

!=

Expected oracle

The expected oracle is never loaded as application state. The comparator evaluates the complete actual persisted and derived result.

Recorded verification

15/15
G00-G14
acceptance gates

185
full regression
tests passed

6/6
final verification
groups

6/6
IT-16 families
attempted and rejected

Repeated verification produced byte-identical evidence.

Reproducibility and governance

requirements.lock

Alembic from empty DB

GitHub Actions verify

evidence SHA-256 manifest

frozen-architecture hashes

protected main

These controls prove deterministic conformance to the bounded reference case - not enterprise or production readiness.

Supported claims, explicit limitations and applicability

The strongest package is the one that states both what was proven and what was deliberately not claimed.

SUPPORTED

- Deterministic execution of frozen Scenarios A-C.
- Explicit business, information and decision semantics.
- Case-local lineage and immutable historical reconstruction.
- Separation of Gate B, Authorisation Eligibility, authority sufficiency and Decision.
- Explicit terminal authority action before Decision persistence.
- Independent oracle comparison, integrity controls and committed evidence.
- Controlled delivery through bounded increments, acceptance gates and reproducibility governance.

NOT CLAIMED

- Company-specific PLM process, data model, workflow or authority fidelity.
- Enterprise PLM completeness or production deployment readiness.
- General arbitrary-graph impact discovery.
- Automated engineering judgement or terminal approval.
- Enterprise programme, budget, workforce-capacity or organisational authority.
- Downstream engineering release, manufacturing, stock or service implementation.

Reusable method: bound the problem -> freeze the meaning -> implement only what the architecture requires -> prove conformance through deterministic evidence.